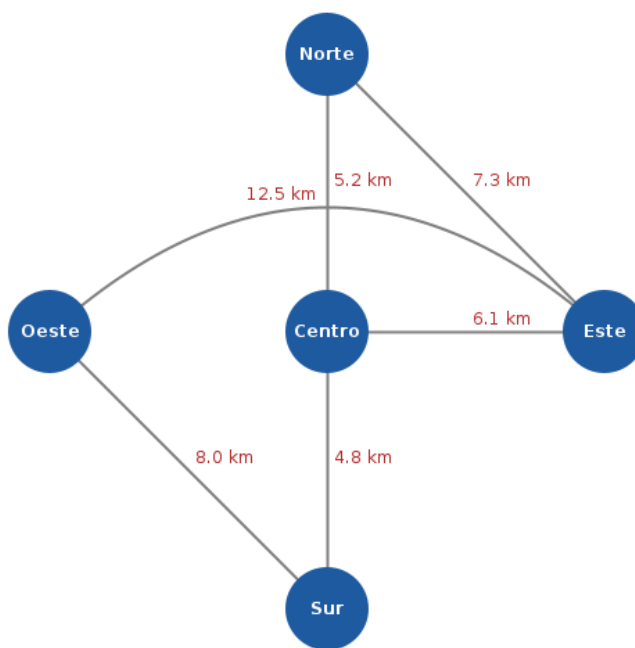


GUÍA COMPLETA DEL PROYECTO

Grafos con ArrayList

Rutas entre zonas de una ciudad



Proyecto **GrafoRutasCiudad** — Estructuras de Datos · Java · NetBeans

Explicación detallada de cada clase, cada método y cómo funciona el programa

Contenido

1. Conceptos básicos antes del código	4
1.1 ¿Qué es un grafo?	4
1.2 Grafo no dirigido	4
1.3 Vecinos y lista de adyacencia	4
1.4 El grafo de la actividad	5
1.5 ¿Qué es un ArrayList?	6
2. Estructura del proyecto	6
2.1 Diagrama de clases	7
2.2 ¿Cómo se guardan los datos en memoria?	8
3. Clase Vertice	8
3.1 Código completo	8
3.2 Explicación parte por parte	9
4. Clase Arista	10
4.1 Código completo	10
4.2 Explicación parte por parte	11
5. Clase Grafo (el corazón del proyecto)	13
5.1 Atributos y constructor	13
5.2 agregarVertice(String nombre) — Pauta 9	13
5.3 buscarVertice(String nombre) — Pauta 11	14
5.4 agregarArista(origen, destino, km) — Pautas 10 y 15	15
5.5 existeArista(Vertice a, Vertice b)	16
5.6 obtenerVecinos(String nombre) — Pauta 12	16
5.7 mostrarVecinos(String nombre) — Pautas 12 y 15	17
5.8 mostrarListaAdyacencia() — Pauta 13	18
5.9 mostrarRutas() — Reto adicional	18
5.10 mostrarVertices(), getVertices() y getAristas()	19
6. Los tres tipos de recorrido (Pauta 14)	19
7. Clase Main (menú interactivo)	20
7.1 Código completo	20
7.2 Flujo del programa	22
7.3 Opciones del menú	23
7.4 Métodos auxiliares de lectura	23

7.5 abrirVentana(grafo)	23
8. Parte gráfica con Swing (reto opcional)	24
8.1 VentanaGrafo	24
8.2 PanelGrafo	24
9. Ejemplo de ejecución	26
9.1 ¿Qué pasa por dentro al elegir la opción 3?	27
10. Cumplimiento de la actividad	27
11. Preguntas frecuentes para la exposición	28
Anexo. Código completo de Grafo y PanelGrafo	30
Grafo.java	30
PanelGrafo.java	33

1. Conceptos básicos antes del código

Antes de leer el código es necesario tener claros los conceptos de la teoría de grafos que usa el programa. Toda la actividad gira alrededor de ellos.

1.1 ¿Qué es un grafo?

Un **grafo** es una estructura de datos formada por un conjunto de **vértices** (también llamados nodos) y un conjunto de **aristas** (también llamadas conexiones o enlaces) que unen pares de vértices. Sirve para representar cualquier situación en la que hay "cosas" relacionadas entre sí: ciudades y carreteras, personas y amistades, computadores y cables de red, etc.

En esta actividad, una empresa de domicilios necesita representar las rutas entre cinco zonas de una ciudad:

Concepto	En el problema	En el código
Vértice	Una zona de la ciudad (Centro, Norte, Sur, Este, Oeste)	Un objeto de la clase <code>Vertice</code>
Arista	Una ruta directa entre dos zonas	Un objeto de la clase <code>Arista</code>
Peso de la arista	La distancia de la ruta en kilómetros	El atributo <code>distanciaKm</code>
Grafo	El mapa completo de zonas y rutas	Un objeto de la clase <code>Grafo</code>

1.2 Grafo no dirigido

Un grafo es **no dirigido** cuando las aristas no tienen sentido: si existe una ruta entre Centro y Norte, se puede ir de Centro a Norte *y también* de Norte a Centro. Es como una calle de doble vía.

Consecuencias importantes en el código:

- Cada ruta se guarda **una sola vez**. La arista Centro-Norte no se guarda dos veces.
- Al buscar si una conexión ya existe se revisan **ambos sentidos** (método `conecta`).
- Si Centro es vecino de Norte, entonces Norte también es vecino de Centro.

En un grafo **dirigido** (que NO es nuestro caso) las aristas serían como calles de una sola vía y habría que guardar A→B y B→A por separado.

1.3 Vecinos y lista de adyacencia

Dos vértices son **vecinos** (o **adyacentes**) si hay una arista que los une directamente. La **lista de adyacencia** es una forma de mostrar el grafo en la que, para cada vértice, se escribe la lista de sus vecinos. Para nuestro grafo es:

```
Centro -> Norte, Sur, Este
Norte  -> Centro, Este
Sur    -> Centro, Oeste
Este   -> Centro, Norte, Oeste
Oeste  -> Este, Sur
```

Observe que las 6 aristas aparecen dos veces en total (por ejemplo, Centro-Norte aparece en la línea de Centro y en la de Norte). Eso es exactamente lo que se espera en un grafo no dirigido.

1.4 El grafo de la actividad

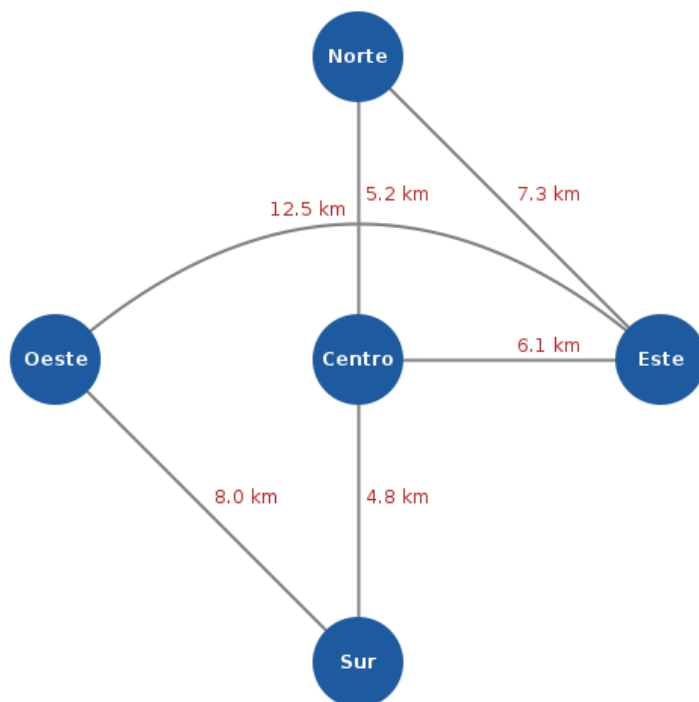


Figura 1. Grafo de la actividad tal como lo dibuja la ventana Swing del programa.

#	Conexión (arista)	Distancia
1	Centro - Norte	5.2 km
2	Centro - Sur	4.8 km
3	Centro - Este	6.1 km
4	Norte - Este	7.3 km
5	Este - Oeste	12.5 km
6	Sur - Oeste	8.0 km

Nota: las distancias en kilómetros son valores de ejemplo (el enunciado no da valores). Puede cambiarlas libremente en el método `cargarDatosIniciales` de la clase `Main`.

1.5 ¿Qué es un ArrayList?

`ArrayList` es una clase de Java (paquete `java.util`) que funciona como un arreglo que **crece automáticamente**. A diferencia de un arreglo normal (`Vertice[]`), no hay que indicar el tamaño al crearlo. Las operaciones que usa este proyecto son:

Operación	Qué hace	Ejemplo en el proyecto
<code>new ArrayList<>()</code>	Crea una lista vacía	<code>vertices = new ArrayList<>();</code>
<code>add(x)</code>	Agrega x al final de la lista	<code>vertices.add(new Vertice(...))</code>
<code>get(i)</code>	Devuelve el elemento en la posición i	<code>aristas.get(i)</code>
<code>size()</code>	Cantidad de elementos	<code>aristas.size()</code>
<code>isEmpty()</code>	true si la lista no tiene elementos	<code>vecinos.isEmpty()</code>
<code>indexOf(x)</code>	Posición de x en la lista (usa equals)	<code>vertices.indexOf(a.getOrigen())</code>
<code>forEach(...)</code>	Ejecuta una acción por cada elemento	<code>aristas.forEach(a -> ...)</code>

El texto entre `< >` se llama **tipo genérico**: `ArrayList<Vertice>` significa "una lista que solo puede contener objetos `Vertice`". Así el compilador impide meter, por error, otra cosa en la lista.

2. Estructura del proyecto

El proyecto es de tipo **Java with Maven** en NetBeans. El código está organizado en cuatro paquetes, cada uno con una responsabilidad distinta:

GrafoRutasCiudad/	
├ pom.xml	(configuración de Maven)
└ src/main/java/	
└ com/grafos/	
└ modelo/	
├ Vertice.java	una zona de la ciudad
└ Arista.java	una ruta entre dos zonas (+ km)
└ estructura/	
└ Grafo.java	administra los dos ArrayList
└ vista/	
├ PanelGrafo.java	dibuja el grafo (Swing)
└ VentanaGrafo.java	ventana que contiene el panel
└ principal/	
└ Main.java	menú y punto de entrada

Paquete	Responsabilidad	Clases
<code>modelo</code>	Representa los datos del problema. No imprime nada ni sabe de menús.	<code>Vertice</code> , <code>Arista</code>
<code>estructura</code>	Contiene la lógica del grafo: agregar, buscar, validar, recorrer.	<code>Grafo</code>

Paquete	Responsabilidad	Clases
vista	Se encarga solo de la parte gráfica (dibujar).	PanelGrafo, VentanaGrafo
principal	Es el punto de entrada : carga los datos y muestra el menú al usuario.	Main

Esta separación es lo que el enunciado llama **separación de responsabilidades**: si mañana se quiere cambiar el dibujo, solo se toca el paquete **vista**; si se quiere cambiar el menú, solo **Main**; la lógica del grafo no se ve afectada.

2.1 Diagrama de clases

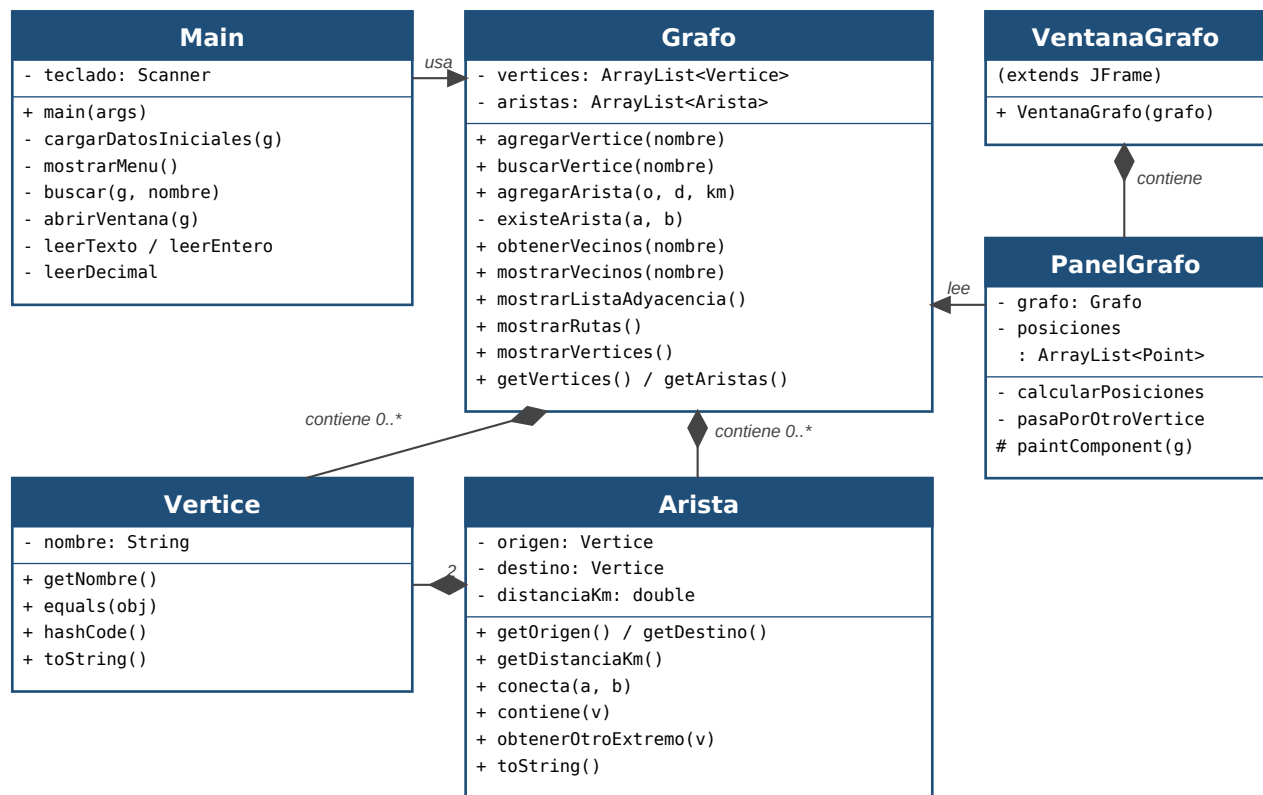


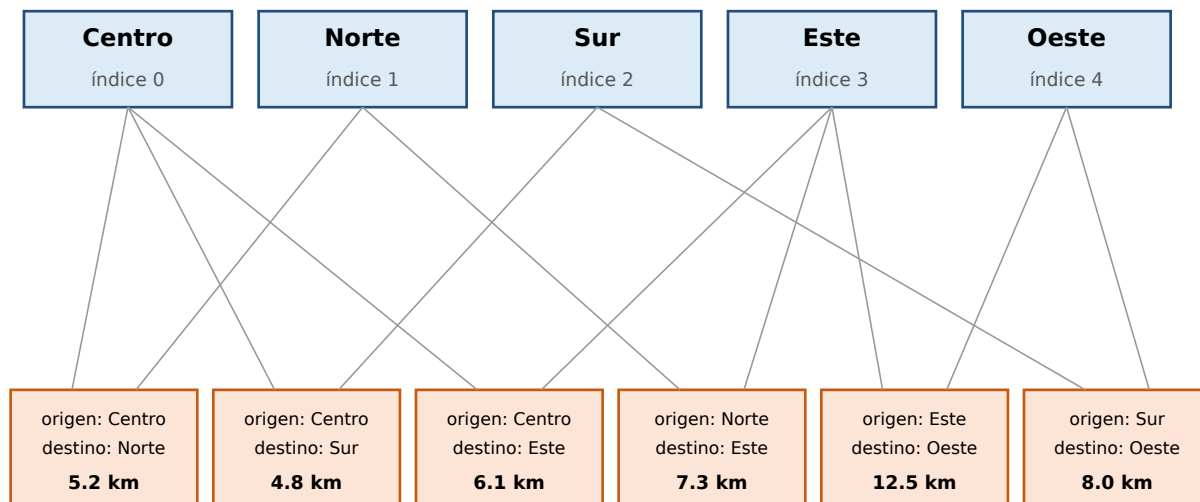
Figura 2. Diagrama de clases simplificado. "-" significa private, "+" public y "#" protected. El rombo indica que una clase contiene objetos de la otra.

- **Main usa Grafo**: crea un grafo y llama a sus métodos según lo que elija el usuario.
- **Grafo contiene** muchos Vertice y muchas Arista (en sus dos ArrayList).
- **Arista contiene 2 Vertice**: su origen y su destino.
- **VentanaGrafo contiene un PanelGrafo**, y el panel lee el grafo para dibujarlo.

2.2 ¿Cómo se guardan los datos en memoria?

Esta es la pregunta clave de la exposición. El grafo tiene dos listas. La lista **vertices** guarda los 5 objetos Vertice. La lista **aristas** guarda 6 objetos Arista, y **cada arista apunta (tiene una referencia) a dos de los objetos Vertice que ya están en la primera lista**:

`ArrayList<Vertice> vertices`



`ArrayList<Arista> aristas`

Figura 3. Contenido de los dos ArrayList después de cargar los datos. Cada línea gris es una referencia de una arista hacia un vértice.

Importante: las aristas **no guardan texto** como "Centro-Norte". Guardan referencias a los mismos objetos Vertice que están en la lista de vértices. No existen dos objetos "Centro": hay uno solo, y todas las aristas que tocan Centro apuntan a él. Esto cumple la condición "No almacenar las conexiones como simples cadenas de texto; deben utilizarse objetos".

3. Clase Vertice

Ubicación: `com.grafos.modelo.Vertice`. Representa **una zona de la ciudad**. Es la clase más sencilla del proyecto: solo guarda un nombre.

3.1 Código completo

```
package com.grafos.modelo;

/**
 * Representa una zona de la ciudad (un vértice del grafo).
 */
public class Vertice {

    private String nombre;

    public Vertice(String nombre) {
```

```

        this.nombre = nombre;
    }

    public String getNombre() {
        return nombre;
    }

    /**
     * Dos vértices son iguales si tienen el mismo nombre (sin importar mayúsculas).
     */
    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (!(obj instanceof Vertice)) {
            return false;
        }
        Vertice otro = (Vertice) obj;
        return nombre.equalsIgnoreCase(otro.nombre);
    }

    @Override
    public int hashCode() {
        return nombre.toLowerCase().hashCode();
    }

    @Override
    public String toString() {
        return nombre;
    }
}

```

3.2 Explicación parte por parte

Atributo y constructor

```

private String nombre;

public Vertice(String nombre) {
    this.nombre = nombre;
}

```

- El atributo es **private**: nadie fuera de la clase puede modificarlo directamente. Esto es **encapsulamiento**.
- El constructor recibe el nombre y lo guarda. **this.nombre** se refiere al atributo del objeto y **nombre** al parámetro recibido.
- No hay método **setNombre**: una vez creada la zona, su nombre no cambia. Así se evita que alguien cambie un nombre y cree duplicados.

getNombre()

Método de acceso (getter). Es la única forma de leer el nombre desde afuera.

equals(Object obj)

Por defecto, en Java dos objetos solo son "iguales" si son exactamente el mismo objeto en memoria. Aquí se **sobrescribe** (`@Override`) para que dos vértices se consideren iguales si tienen el **mismo nombre, sin importar mayúsculas**:

- `if (this == obj) return true;` — si es el mismo objeto, obviamente son iguales.
- `if (!(obj instanceof Vertice)) return false;` — si lo que se compara no es un `Vertice`, no son iguales.
- `return nombre.equalsIgnoreCase(otro.nombre);` — compara los nombres ignorando mayúsculas: "Centro", "centro" y "CENTRO" son la misma zona.

¿Dónde se usa? En `Arista.conecta()`, `Arista.contiene()`, `Arista.obtenerOtroExtremo()` y en `ArrayList.indexOf()` dentro del panel de dibujo.

hashCode()

Regla de Java: si se sobrescribe `equals`, también se debe sobrescribir `hashCode`, de forma que dos objetos iguales produzcan el mismo código. Por eso se calcula sobre el nombre en minúsculas. En este proyecto no se usa directamente, pero es una buena práctica obligatoria.

toString()

Define cómo se ve el objeto cuando se imprime. Gracias a este método, `System.out.println(vertice)` muestra "Centro" y no algo como `com.grafos.modelo.Vertice@1b6d3586`. También hace que imprimir una lista de vecinos muestre [Norte, Sur, Este].

4. Clase Arista

Ubicación: `com.grafos.modelo.Arista`. Representa **una ruta entre dos zonas**, con su distancia en kilómetros (reto adicional).

4.1 Código completo

```
package com.grafos.modelo;

/**
 * Representa una conexión (ruta) entre dos zonas de la ciudad.
 * Como el grafo es NO dirigido, la ruta sirve en ambos sentidos.
 */
public class Arista {

    private Vertice origen;
    private Vertice destino;
    private double distanciaKm;

    public Arista(Vertice origen, Vertice destino, double distanciaKm) {
        this.origen = origen;
        this.destino = destino;
        this.distanciaKm = distanciaKm;
    }

    public Vertice getOrigen() {
```

```

        return origen;
    }

    public Vertice getDestino() {
        return destino;
    }

    public double getDistanciaKm() {
        return distanciaKm;
    }

    /**
     * Indica si esta arista une a los vértices a y b (en cualquier sentido).
     */
    public boolean conecta(Vertice a, Vertice b) {
        return (origen.equals(a) && destino.equals(b))
            || (origen.equals(b) && destino.equals(a));
    }

    /**
     * Indica si el vértice v es uno de los extremos de esta arista.
     */
    public boolean contiene(Vertice v) {
        return origen.equals(v) || destino.equals(v);
    }

    /**
     * Devuelve el vértice del otro extremo (el vecino de v).
     * Si v no pertenece a la arista, devuelve null.
     */
    public Vertice obtenerOtroExtremo(Vertice v) {
        if (origen.equals(v)) {
            return destino;
        }
        if (destino.equals(v)) {
            return origen;
        }
        return null;
    }

    @Override
    public String toString() {
        return origen.getNombre() + " - " + destino.getNombre()
            + " : " + distanciaKm + " km";
    }
}

```

4.2 Explicación parte por parte

Atributos

Atributo	Tipo	Significado
origen	Vertice	Uno de los extremos de la ruta (el primero que se indicó)
destino	Vertice	El otro extremo de la ruta
distanciaKm	double	Longitud de la ruta en kilómetros. Es double porque puede tener decimales (5.2)

Aunque se llamen "origen" y "destino", como el grafo es no dirigido **el orden no importa**: la arista Centro-Norte sirve igual para ir de Norte a Centro. Los nombres solo indican cuál se

escribió primero.

conecta(Vertex a, Vertex b)

```
/**
 * Indica si esta arista une a los vértices a y b (en cualquier sentido).
 */
public boolean conecta(Vertex a, Vertex b) {
    return (origen.equals(a) && destino.equals(b))
        || (origen.equals(b) && destino.equals(a));
}
```

Responde: ¿esta arista une a los vértices a y b? Revisa los dos sentidos posibles, unidos con || (O lógico):

Arista guardada	Llamada	(origen=a y destino=b)	(origen=b y destino=a)	Resultado
Centro-Norte	conecta(Centro, Norte)	true	—	true
Centro-Norte	conecta(Norte, Centro)	false	true	true
Centro-Norte	conecta(Centro, Sur)	false	false	false

Gracias a este método el programa no deja crear Norte-Centro si ya existe Centro-Norte.

contiene(Vertex v)

Responde: ¿el vértice v es uno de los dos extremos de esta arista? Se usa para encontrar vecinos: todas las aristas que "contienen" a Centro son rutas que salen de Centro.

obtenerOtroExtremo(Vertex v)

```
/**
 * Devuelve el vértice del otro extremo (el vecino de v).
 * Si v no pertenece a la arista, devuelve null.
 */
public Vertex obtenerOtroExtremo(Vertex v) {
    if (origen.equals(v)) {
        return destino;
    }
    if (destino.equals(v)) {
        return origen;
    }
    return null;
}
```

Dado uno de los extremos, devuelve el otro. Es la pieza clave para calcular vecinos:

- En la arista Centro-Norte, **obtenerOtroExtremo(Centro)** devuelve **Norte**.
- En la misma arista, **obtenerOtroExtremo(Norte)** devuelve **Centro**.
- Si se pasa un vértice que no está en la arista (por ejemplo Sur), devuelve **null**.

toString()

Muestra la arista en formato legible, por ejemplo: **Centro - Norte : 5.2 km.**

5. Clase Grafo (el corazón del proyecto)

Ubicación: `com.grafos.estructura.Grafo`. Es la clase que **administra toda la estructura**: guarda los dos `ArrayList` y tiene todas las operaciones que pide la actividad.

5.1 Atributos y constructor

```
private ArrayList<Vertice> vertices;  
private ArrayList<Arista> aristas;  
  
public Grafo() {  
    vertices = new ArrayList<>();  
    aristas = new ArrayList<>();  
}
```

Las dos listas son **private**. Nadie puede agregar ni borrar elementos directamente desde afuera; hay que pasar por los métodos de `Grafo`, que **validan** los datos. El constructor crea ambas listas vacías.

Método	Pauta	Qué hace
<code>agregarVertice</code>	9	Agrega una zona evitando nombres repetidos
<code>buscarVertice</code>	11	Busca una zona por su nombre
<code>agregarArista</code>	10, 15	Crea una ruta entre dos zonas existentes, con validaciones
<code>existeArista</code>	10	Auxiliar privado: ¿ya existe esa conexión?
<code>obtenerVecinos</code>	12	Devuelve la lista de vecinos de una zona
<code>mostrarVecinos</code>	12, 15	Imprime los vecinos o un error si la zona no existe
<code>mostrarListaAdyacencia</code>	13	Imprime la lista de adyacencia completa
<code>mostrarRutas</code>	Reto	Imprime origen, destino y distancia de cada ruta
<code>mostrarVertices</code>	—	Imprime todas las zonas
<code>getVertices</code> / <code>getAristas</code>	—	Devuelven copias de las listas (para el dibujo)

5.2 agregarVertice(String nombre) — Pauta 9

```
/**  
 * Agrega un vértice nuevo evitando nombres repetidos.  
 */  
public boolean agregarVertice(String nombre) {  
    if (nombre == null || nombre.trim().isEmpty()) {  
        System.out.println("Error: el nombre del vértice no puede estar vacío.");  
        return false;  
    }  
}
```

```

    if (buscarVertice(nombre) != null) {
        System.out.println("Error: el vértice '" + nombre + "' ya existe.");
        return false;
    }
    vertices.add(new Vertice(nombre.trim()));
    System.out.println("Vértice '" + nombre.trim() + "' agregado.");
    return true;
}

```

Funciona como un filtro con tres pasos:

- **1. Nombre vacío:** si el nombre es `null` o solo tiene espacios (`trim()` quita los espacios de los extremos y `isEmpty()` revisa si quedó vacío), muestra un error y devuelve `false`.
- **2. Nombre repetido:** usa `buscarVertice`. Si encuentra una zona con ese nombre (no devuelve `null`), muestra un error y devuelve `false`. Como la búsqueda ignora mayúsculas, tampoco deja agregar "centro" si ya existe "Centro".
- **3. Todo bien:** crea un nuevo objeto `Vertice` y lo agrega a la lista con `add`. Devuelve `true`.

El valor `boolean` devuelto permite saber si la operación funcionó, aunque en este programa basta con el mensaje en pantalla.

5.3 buscarVertice(String nombre) — Pauta 11

```

/**
 * Busca un vértice por su nombre (recorrido con for-each).
 * Devuelve null si no existe.
 */
public Vertice buscarVertice(String nombre) {
    if (nombre == null) {
        return null;
    }
    for (Vertice v : vertices) {
        if (v.getNombre().equalsIgnoreCase(nombre.trim())) {
            return v;
        }
    }
    return null;
}

```

Es una **búsqueda lineal**: recorre la lista de vértices uno por uno con un **for-each** y compara nombres. Si encuentra coincidencia, devuelve ese objeto de inmediato (`return v`). Si termina el recorrido sin encontrarlo, devuelve `null`, que significa "no existe".

Ejemplo: `buscarVertice("este")`:

Vuelta	v	¿"Centro".equalsIgnoreCase("este")?	Acción
1	Centro	false	sigue
2	Norte	false	sigue
3	Sur	false	sigue
4	Este	true	return Este (termina)

Este método es muy importante porque lo usan casi todos los demás: `agregarVertice`, `agregarArista`, `obtenerVecinos` y `mostrarVecinos`.

5.4 `agregarArista(origen, destino, km)` — Pautas 10 y 15

```
/**
 * Agrega una arista entre dos vértices existentes.
 */
public boolean agregarArista(String nombreOrigen, String nombreDestino, double distanciaKm) {
    Vertice origen = buscarVertice(nombreOrigen);
    Vertice destino = buscarVertice(nombreDestino);

    if (origen == null) {
        System.out.println("Error: no se puede conectar, el vértice '" + nombreOrigen + "' no existe.");
        return false;
    }
    if (destino == null) {
        System.out.println("Error: no se puede conectar, el vértice '" + nombreDestino + "' no existe.");
        return false;
    }
    if (origen.equals(destino)) {
        System.out.println("Error: no se puede conectar un vértice consigo mismo (" + nombreOrigen + ").");
        return false;
    }
    if (distanciaKm <= 0) {
        System.out.println("Error: la distancia debe ser mayor que 0 km.");
        return false;
    }
    if (existeArista(origen, destino)) {
        System.out.println("Error: la conexión " + origen + " - " + destino + " ya existe.");
        return false;
    }
    aristas.add(new Arista(origen, destino, distanciaKm));
    System.out.println("Conexión " + origen + " - " + destino + " (" + distanciaKm + " km) agregada.");
    return true;
}
```

Primero convierte los **nombres** (texto) en **objetos** `Vertice` usando `buscarVertice`. Luego aplica cinco validaciones en orden; si alguna falla, muestra un mensaje claro y termina con `return false`:

#	Validación	Ejemplo que falla	Mensaje
1	El origen existe	Aeropuerto – Norte	no se puede conectar, el vértice 'Aeropuerto' no existe
2	El destino existe	Centro – Aeropuerto	no se puede conectar, el vértice 'Aeropuerto' no existe
3	No es el mismo vértice	Sur – Sur	no se puede conectar un vértice consigo mismo
4	Distancia mayor que 0	Centro – Norte, -3 km	la distancia debe ser mayor que 0 km

#	Validación	Ejemplo que falla	Mensaje
5	La conexión no existe ya	Norte - Centro	la conexión Norte - Centro ya existe

Si pasa todas las validaciones, crea el objeto `new Arista(origen, destino, distanciaKm)` y lo agrega a la lista `aristas`. Observe que la arista recibe los **objetos** encontrados, no los textos.

Traza: agregarArista("Norte", "Centro", 5.2)

- origen = buscarVertice("Norte") → objeto Norte ✓
- destino = buscarVertice("Centro") → objeto Centro ✓
- ¿Norte es igual a Centro? No ✓
- ¿5.2 <= 0? No ✓
- existeArista(Norte, Centro) → recorre las aristas; la primera es Centro-Norte y `conecta(Norte, Centro)` da true (sentido inverso) → **true X**
- Resultado: "Error: la conexión Norte - Centro ya existe." y devuelve false.

5.5 existeArista(Vertice a, Vertice b)

```
/**
 * Verifica si ya existe una arista entre a y b (recorrido con for clásico).
 */
private boolean existeArista(Vertice a, Vertice b) {
    for (int i = 0; i < aristas.size(); i++) {
        if (aristas.get(i).conecta(a, b)) {
            return true;
        }
    }
    return false;
}
```

Método auxiliar **privado** (solo lo usa Grafo por dentro). Recorre las aristas con un **for clásico** usando el índice `i` y `get(i)`. Si alguna arista conecta a y b en cualquier sentido, devuelve `true`. Se hizo privado porque es un detalle interno: desde afuera nadie necesita llamarlo.

5.6 obtenerVecinos(String nombre) — Pauta 12

```
/**
 * Devuelve la lista de vecinos de un vértice.
 * Si el vértice no existe, devuelve una lista vacía.
 */
public ArrayList<Vertice> obtenerVecinos(String nombre) {
    ArrayList<Vertice> vecinos = new ArrayList<>();
    Vertice v = buscarVertice(nombre);
    if (v == null) {
```

```

        return vecinos;
    }
    for (Arista a : aristas) {
        if (a.contiene(v)) {
            vecinos.add(a.obtenerOtroExtremo(v));
        }
    }
    return vecinos;
}

```

Este es el método que "entiende" el grafo. Algoritmo:

- Crea una lista vacía **vecinos**.
- Busca el vértice. Si no existe, devuelve la lista vacía.
- Recorre **todas** las aristas con for-each. Por cada arista que contiene al vértice, toma el otro extremo y lo agrega a **vecinos**.
- Devuelve la lista.

Traza: obtenerVecinos("Centro")

Arista	¿contiene Centro?	Otro extremo	vecinos después
Centro - Norte	sí	Norte	[Norte]
Centro - Sur	sí	Sur	[Norte, Sur]
Centro - Este	sí	Este	[Norte, Sur, Este]
Norte - Este	no	—	[Norte, Sur, Este]
Este - Oeste	no	—	[Norte, Sur, Este]
Sur - Oeste	no	—	[Norte, Sur, Este]

Resultado: **[Norte, Sur, Este]**. Para Oeste funcionaría igual: en Este-Oeste el otro extremo es Este, y en Sur-Oeste es Sur, así que el resultado es [Este, Sur].

Idea clave: los vecinos **no se guardan** en ningún lado; se **calculan** cada vez recorriendo la lista de aristas. Por eso, si se agrega una ruta nueva, los vecinos se actualizan solos.

5.7 mostrarVecinos(String nombre) — Pautas 12 y 15

```

/**
 * Muestra en consola los vecinos de un vértice.
 */
public void mostrarVecinos(String nombre) {
    Vertice v = buscarVertice(nombre);
    if (v == null) {
        System.out.println("Error: el vértice '" + nombre + "' no existe.");
        return;
    }
}

```

```

    }
    ArrayList<Vertice> vecinos = obtenerVecinos(nombre);
    if (vecinos.isEmpty()) {
        System.out.println(v + " no tiene vecinos.");
    } else {
        System.out.println("Vecinos de " + v + ": " + vecinos);
    }
}

```

Es la versión "para el usuario" de obtenerVecinos. Distingue tres casos y muestra un mensaje para cada uno: la zona no existe (error), la zona existe pero no tiene rutas, o la zona tiene vecinos. Separar **obtenerVecinos** (calcula) de **mostrarVecinos** (imprime) es otro ejemplo de separación de responsabilidades: el cálculo se puede reutilizar sin imprimir nada.

5.8 mostrarListaAdyacencia() — Pauta 13

```

/**
 * Muestra la lista de adyacencia del grafo.
 */
public void mostrarListaAdyacencia() {
    System.out.println("\n===== LISTA DE ADYACENCIA =====");
    for (Vertice v : vertices) {
        ArrayList<Vertice> vecinos = obtenerVecinos(v.getNombre());
        StringBuilder linea = new StringBuilder(v.getNombre() + " -> ");
        for (int i = 0; i < vecinos.size(); i++) {
            linea.append(vecinos.get(i).getNombre());
            if (i < vecinos.size() - 1) {
                linea.append(", ");
            }
        }
        System.out.println(linea);
    }
}

```

Por cada vértice (for-each externo) obtiene sus vecinos y arma una línea de texto. Usa un **for clásico** interno porque necesita saber la posición **i**: la coma solo se agrega si el vecino **no es el último** (**i < vecinos.size() - 1**). Así se evita una coma sobrante al final ("Centro -> Norte, Sur, Este," estaría mal).

StringBuilder es una clase para construir textos por partes de forma eficiente: se va agregando con **append** y al final se imprime.

5.9 mostrarRutas() — Reto adicional

```

/**
 * Muestra todas las rutas con origen, destino y distancia (recorrido con forEach).
 */
public void mostrarRutas() {
    System.out.println("\n===== RUTAS DISPONIBLES =====");
    if (aristas.isEmpty()) {
        System.out.println("No hay rutas registradas.");
        return;
    }
}

```

```

    }
    aristas.forEach(a -> System.out.println(
        "Origen: " + a.getOrigen()
        + " | Destino: " + a.getDestino()
        + " | Distancia: " + a.getDistanciaKm() + " km"));
}

```

Si no hay aristas, avisa. Si hay, las recorre con **forEach** y una **expresión lambda**. La lambda `a -> System.out.println(...)` se lee: "para cada arista *a*, imprime su origen, destino y distancia". Es una forma más corta de escribir un for-each.

5.10 mostrarVertices(), getVertices() y getAristas()

```

// Se devuelven copias para proteger las listas internas (encapsulamiento).
public ArrayList<Vertice> getVertices() {
    return new ArrayList<>(vertices);
}

public ArrayList<Arista> getAristas() {
    return new ArrayList<>(aristas);
}

```

Los getters no devuelven la lista original sino una **copia** (`new ArrayList<>(vertices)`). Si alguien hace `grafo.getVertices().add(...)` o `.clear()`, solo modifica la copia; el grafo real queda intacto. Así nadie puede saltarse las validaciones de `agregarVertice`. Esto es **encapsulamiento**. Los usa el panel de dibujo para saber qué dibujar.

6. Los tres tipos de recorrido (Pauta 14)

La pauta 14 pide recorrer los ArrayList con **for**, **for-each** o **forEach**. El proyecto usa los tres:

Tipo	Sintaxis	Cuándo conviene	Dónde se usa
for clásico	<code>for (int i = 0; i < lista.size(); i++)</code>	Cuando se necesita la posición (índice) del elemento	<code>existeArista</code> , <code>mostrarListaAdyacencia</code> (comas), <code>PanelGrafo</code>
for-each	<code>for (Vertice v : vertices)</code>	Cuando solo se necesita cada elemento, sin su posición	<code>buscarVertice</code> , <code>obtenerVecinos</code> , <code>mostrarListaAdyacencia</code>
forEach + lambda	<code>lista.forEach(x -> ...)</code>	Para acciones cortas sobre cada elemento (por ejemplo imprimir)	<code>mostrarRutas</code> , <code>mostrarVertices</code>

Los tres hacen lo mismo en el fondo (visitar cada elemento de la lista); cambia la forma de escribirlo. Ejemplo equivalente con las tres formas:

```
// for clásico
for (int i = 0; i < vertices.size(); i++) {
    System.out.println(vertices.get(i));
}

// for-each
for (Vertice v : vertices) {
    System.out.println(v);
}

// forEach con lambda
vertices.forEach(v -> System.out.println(v));
```

7. Clase Main (menú interactivo)

Ubicación: `com.grafos.principal.Main`. Es el **punto de entrada** del programa (tiene el método `main`). No contiene lógica del grafo: solo carga los datos, muestra el menú, lee lo que escribe el usuario y llama a los métodos de Grafo.

7.1 Código completo

```
package com.grafos.principal;

import com.grafos.estructura.Grafo;
import com.grafos.modelo.Vertice;
import com.grafos.vista.VentanaGrafo;
import java.awt.GraphicsEnvironment;
import java.util.Scanner;
import javax.swing.SwingUtilities;

public class Main {

    private static Scanner teclado = new Scanner(System.in);

    public static void main(String[] args) {
        Grafo grafo = new Grafo();
        cargarDatosIniciales(grafo);

        int opcion;
        do {
            mostrarMenu();
            opcion = leerEntero("Elija una opción: ");

            switch (opcion) {
                case 1:
                    grafo.mostrarVertices();
                    break;
                case 2:
                    buscar(grafo, leerTexto("Nombre de la zona a buscar: "));
                    break;
                case 3:
                    grafo.mostrarVecinos(leerTexto("Nombre de la zona: "));
                    break;
                case 4:
                    grafo.mostrarListaAdyacencia();
                    break;
                case 5:
                    grafo.mostrarRutas();
                    break;
                case 6:

```

```

        grafo.agregarVertice(leerTexto("Nombre de la nueva zona: "));
        break;
    case 7:
        String origen = leerTexto("Zona de origen: ");
        String destino = leerTexto("Zona de destino: ");
        double km = leerDecimal("Distancia en km: ");
        grafo.agregarArista(origen, destino, km);
        break;
    case 8:
        abrirVentana(grafo);
        break;
    case 0:
        System.out.println("Programa finalizado.");
        break;
    default:
        System.out.println("Error: opción no válida.");
    }
} while (opcion != 0);

System.exit(0); // Cierra también las ventanas Swing que sigan abiertas
}

/**
 * Registra las 5 zonas y las 6 conexiones pedidas en la actividad.
 */
private static void cargarDatosIniciales(Grafo grafo) {
    System.out.println("==== REGISTRO DE ZONAS =====");
    grafo.agregarVertice("Centro");
    grafo.agregarVertice("Norte");
    grafo.agregarVertice("Sur");
    grafo.agregarVertice("Este");
    grafo.agregarVertice("Oeste");

    System.out.println("\n==== REGISTRO DE CONEXIONES =====");
    grafo.agregarArista("Centro", "Norte", 5.2);
    grafo.agregarArista("Centro", "Sur", 4.8);
    grafo.agregarArista("Centro", "Este", 6.1);
    grafo.agregarArista("Norte", "Este", 7.3);
    grafo.agregarArista("Este", "Oeste", 12.5);
    grafo.agregarArista("Sur", "Oeste", 8.0);
}

private static void mostrarMenu() {
    System.out.println("\n===== MENÚ =====");
    System.out.println("1. Mostrar zonas");
    System.out.println("2. Buscar una zona");
    System.out.println("3. Ver vecinos de una zona");
    System.out.println("4. Mostrar lista de adyacencia");
    System.out.println("5. Mostrar rutas (origen, destino, distancia)");
    System.out.println("6. Agregar una zona");
    System.out.println("7. Agregar una conexión");
    System.out.println("8. Ver el grafo (ventana gráfica)");
    System.out.println("0. Salir");
}

private static void buscar(Grafo grafo, String nombre) {
    Vertice v = grafo.buscarVertice(nombre);
    if (v != null) {
        System.out.println("Zona encontrada: " + v);
    }
}

```

```

    } else {
        System.out.println("Error: la zona '" + nombre + "' no existe.");
    }
}

private static void abrirVentana(Grafo grafo) {
    if (GraphicsEnvironment.isHeadless()) {
        System.out.println("Error: no hay entorno gráfico disponible.");
        return;
    }
    SwingUtilities.invokeLater(() -> new VentanaGrafo(grafo).setVisible(true));
    System.out.println("Ventana abierta. Puede seguir usando el menú.");
}

private static String leerTexto(String mensaje) {
    System.out.print(mensaje);
    return teclado.nextLine().trim();
}

/**
 * Lee un número entero; si el usuario escribe otra cosa devuelve -1.
 */
private static int leerEntero(String mensaje) {
    try {
        return Integer.parseInt(leerTexto(mensaje));
    } catch (NumberFormatException e) {
        return -1;
    }
}

/**
 * Lee un número decimal (acepta punto o coma); si no es válido devuelve -1.
 */
private static double leerDecimal(String mensaje) {
    try {
        return Double.parseDouble(leerTexto(mensaje).replace(',', '.'));
    } catch (NumberFormatException e) {
        System.out.println("Error: debe escribir un número.");
        return -1;
    }
}
}

```

7.2 Flujo del programa

- **1.** Se crea el objeto **Grafo** (con sus dos listas vacías).
- **2.** **cargarDatosIniciales** registra las 5 zonas (pauta 7) y las 6 conexiones con sus distancias (pauta 8).
- **3.** Empieza un ciclo **do { ... } while (opcion != 0)**: muestra el menú, lee la opción y con un **switch** ejecuta la acción correspondiente.
- **4.** El ciclo se repite hasta que el usuario escribe 0.
- **5.** **System.exit(0)** termina el programa (y cierra la ventana gráfica si quedó abierta).

Se usa **do-while** (y no **while**) porque el menú debe mostrarse **al menos una vez** antes de preguntar si se sale.

7.3 Opciones del menú

Opción	Acción	Método llamado
1	Mostrar zonas	<code>grafo.mostrarVertices()</code>
2	Buscar una zona	<code>buscar(grafo, nombre) → grafo.buscarVertice</code>
3	Ver vecinos de una zona (seleccionada por el usuario)	<code>grafo.mostrarVecinos(nombre)</code>
4	Mostrar lista de adyacencia	<code>grafo.mostrarListaAdyacencia()</code>
5	Mostrar rutas	<code>grafo.mostrarRutas()</code>
6	Agregar una zona	<code>grafo.agregarVertice(nombre)</code>
7	Agregar una conexión	<code>grafo.agregarArista(o, d, km)</code>
8	Ver el grafo en ventana	<code>abrirVentana(grafo)</code>
0	Salir	—

7.4 Métodos auxiliares de lectura

Método	Qué hace
<code>leerTexto(mensaje)</code>	Muestra el mensaje y lee una línea completa con <code>teclado.nextLine()</code> . Usa <code>trim()</code> para quitar espacios sobrantes.
<code>leerEntero(mensaje)</code>	Lee un texto y lo convierte a entero con <code>Integer.parseInt()</code> . Si el usuario escribe letras, se produce una <code>NumberFormatException</code> ; el <code>catch</code> la atrapa y devuelve -1, que el switch trata como "opción no válida". Así el programa no se cae.
<code>leerDecimal(mensaje)</code>	Igual, pero con <code>Double.parseDouble()</code> . Además cambia la coma por punto (<code>replace(',', '.')</code>) para aceptar tanto 5.2 como 5,2. Si no es número devuelve -1, que luego <code>agregarArista</code> rechaza porque la distancia debe ser mayor que 0.

Se lee siempre con `nextLine()` (y no con `nextInt()`) para evitar un problema clásico del Scanner: `nextInt()` deja el "Enter" pendiente y el siguiente `nextLine()` lo toma como una línea vacía.

7.5 abrirVentana(grafo)

Revisa primero si hay pantalla disponible (`GraphicsEnvironment.isHeadless()`). Luego crea la ventana con `SwingUtilities.invokeLater`, que es la forma correcta de crear componentes Swing: los ejecuta en el hilo especial de la interfaz gráfica. Gracias a eso, la ventana y el menú de consola funcionan al mismo tiempo.

8. Parte gráfica con Swing (reto opcional)

8.1 VentanaGrafo

```
package com.grafos.vista;

import com.grafos.estructura.Grafo;
import javax.swing.JFrame;

/**
 * Ventana principal que contiene el panel de dibujo del grafo.
 */
public class VentanaGrafo extends JFrame {

    public VentanaGrafo(Grafo grafo) {
        super("Rutas entre zonas de la ciudad");
        add(new PanelGrafo(grafo));
        pack();
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
    }
}
```

- **extends JFrame**: la clase **es** una ventana (herencia).
- **super("...")**: llama al constructor de JFrame para poner el título.
- **add(new PanelGrafo(grafo))**: coloca dentro el panel que dibuja.
- **pack()**: ajusta el tamaño de la ventana al tamaño preferido del panel (600 × 500).
- **setLocationRelativeTo(null)**: centra la ventana en la pantalla.
- **DISPOSE_ON_CLOSE**: al cerrar la ventana solo se cierra ella, **no todo el programa**; el menú sigue funcionando.

8.2 PanelGrafo

PanelGrafo extends JPanel. Un JPanel es un área rectangular donde se puede dibujar. Java llama automáticamente al método **paintComponent(Graphics g)** cada vez que hay que pintar el panel (al abrir la ventana, al cambiar su tamaño, etc.). Ahí va todo el dibujo.

Atributos y constantes

Elemento	Significado
RADIO = 30	Radio de cada círculo (vértice), en píxeles
CURVATURA = 90	Qué tanto se curva una línea que pasaría por encima de otro vértice
grafo	Referencia al grafo que se va a dibujar
posiciones	ArrayList<Point> : la posición (x, y) de cada vértice en pantalla, en el mismo orden que la lista de vértices

Las posiciones se guardan en el panel y **no** en la clase Vertice, porque son un detalle de dibujo. El modelo no debe depender de la parte gráfica.

calcularPosiciones

```
/**
 * Calcula las posiciones: zonas conocidas en su punto cardinal,
 * cualquier otra zona se ubica en círculo alrededor del centro.
 */
private void calcularPosiciones(ArrayList<Vertice> vertices) {
    posiciones.clear();
    int cx = getWidth() / 2;
    int cy = getHeight() / 2;
    int d = Math.min(getWidth(), getHeight()) / 2 - RADIO - 20;

    for (int i = 0; i < vertices.size(); i++) {
        String nombre = vertices.get(i).getNombre().toLowerCase();
        switch (nombre) {
            case "centro":
                posiciones.add(new Point(cx, cy));
                break;
            case "norte":
                posiciones.add(new Point(cx, cy - d));
                break;
            case "sur":
                posiciones.add(new Point(cx, cy + d));
                break;
            case "este":
                posiciones.add(new Point(cx + d, cy));
                break;
            case "oeste":
                posiciones.add(new Point(cx - d, cy));
                break;
            default:
                double angulo = 2 * Math.PI * i / vertices.size() + Math.PI / 4;
                posiciones.add(new Point(
                    cx + (int) (d * Math.cos(angulo)),
                    cy + (int) (d * Math.sin(angulo))));
        }
    }
}
```

Calcula el centro del panel (**cx**, **cy**) y una distancia **d**. Luego, con un **switch** sobre el nombre, pone cada zona en su punto cardinal: Centro al medio, Norte arriba (y menor), Sur abajo, Este a la derecha y Oeste a la izquierda. Recuerde que en Swing **la coordenada y crece hacia abajo**.

Si el usuario agrega una zona nueva (por ejemplo "Aeropuerto"), cae en **default** y se ubica sobre un círculo usando trigonometría: $x = cx + d \cdot \cos(\text{ángulo})$, $y = cy + d \cdot \sin(\text{ángulo})$.

pasaPorOtroVertice

Revisa si el punto medio de una línea cae encima de algún vértice. Esto pasa con Este-Oeste: la línea recta atravesaría el círculo de Centro. En ese caso la arista se dibuja como **curva** (**QuadCurve2D**) para que se vea y su distancia no quede tapada.

paintComponent: el dibujo

- Llama a **super.paintComponent(g)** para limpiar el fondo.
- Activa el **antialiasing** para que los bordes se vean suaves.
- Pide **grafo.getVertices()** y calcula las posiciones.

- **Primero dibuja las aristas:** por cada arista busca la posición de su origen y su destino con `vertices.indexOf(...)` (usa el `equals` de `Vertice`), traza la línea (o curva) y escribe la distancia en rojo cerca de la mitad.
- **Después dibuja los vértices:** un círculo azul (`fill0val`) con el nombre en blanco, centrado usando `FontMetrics` para medir el ancho del texto.

Se dibujan primero las líneas y después los círculos para que los círculos queden **encima** y tapen los extremos de las líneas.

9. Ejemplo de ejecución

Al iniciar, el programa registra los datos y muestra el menú:

```
===== REGISTRO DE ZONAS =====
Vértice 'Centro' agregado.
Vértice 'Norte' agregado.
Vértice 'Sur' agregado.
Vértice 'Este' agregado.
Vértice 'Oeste' agregado.

===== REGISTRO DE CONEXIONES =====
Conexión Centro - Norte (5.2 km) agregada.
Conexión Centro - Sur (4.8 km) agregada.
Conexión Centro - Este (6.1 km) agregada.
Conexión Norte - Este (7.3 km) agregada.
Conexión Este - Oeste (12.5 km) agregada.
Conexión Sur - Oeste (8.0 km) agregada.

===== MENÚ =====
1. Mostrar zonas
2. Buscar una zona
3. Ver vecinos de una zona
4. Mostrar lista de adyacencia
5. Mostrar rutas (origen, destino, distancia)
6. Agregar una zona
7. Agregar una conexión
8. Ver el grafo (ventana gráfica)
0. Salir
```

Ejemplos de uso (lo que escribe el usuario está después de los dos puntos):

```
Elija una opción: 3
Nombre de la zona: Centro
Vecinos de Centro: [Norte, Sur, Este]

Elija una opción: 3
Nombre de la zona: Aeropuerto
Error: el vértice 'Aeropuerto' no existe.

Elija una opción: 6
```

```

Nombre de la nueva zona: Centro
Error: el vértice 'Centro' ya existe.

Elija una opción: 7
Zona de origen: Norte
Zona de destino: Centro
Distancia en km: 3
Error: la conexión Norte - Centro ya existe.

Elija una opción: 4
===== LISTA DE ADYACENCIA =====
Centro -> Norte, Sur, Este
Norte -> Centro, Este
Sur -> Centro, Oeste
Este -> Centro, Norte, Oeste
Oeste -> Este, Sur

Elija una opción: 5
===== RUTAS DISPONIBLES =====
Origen: Centro | Destino: Norte | Distancia: 5.2 km
Origen: Centro | Destino: Sur | Distancia: 4.8 km
Origen: Centro | Destino: Este | Distancia: 6.1 km
Origen: Norte | Destino: Este | Distancia: 7.3 km
Origen: Este | Destino: Oeste | Distancia: 12.5 km
Origen: Sur | Destino: Oeste | Distancia: 8.0 km

```

9.1 ¿Qué pasa por dentro al elegir la opción 3?

- **Main:** `leerEntero` convierte "3" en el número 3; el `switch` entra al `case 3`.
- **Main:** `leerTexto` lee "Centro" y llama a `grafo.mostrarVecinos("Centro")`.
- **Grafo:** `mostrarVecinos` llama a `buscarVertice("Centro")`, que recorre la lista de vértices y devuelve el objeto Centro.
- **Grafo:** llama a `obtenerVecinos("Centro")`, que recorre las 6 aristas y, usando `contiene` y `obtenerOtroExtremo` de **Arista**, arma la lista [Norte, Sur, Este].
- **Grafo:** imprime la lista. Java usa el `toString` de **Vertice** para mostrar cada nombre.
- **Main:** el ciclo vuelve a mostrar el menú.

En este recorrido participan las cuatro clases principales, cada una haciendo solo su parte.

10. Cumplimiento de la actividad

#	Pauta	Dónde se cumple
1	Proyecto organizado mediante clases	6 clases en 4 paquetes
2	Clase Vertice	<code>modelo/Vertice.java</code>
3	Clase Arista	<code>modelo/Arista.java</code>
4	Clase Grafo	<code>estructura/Grafo.java</code>
5	<code>ArrayList<Vertice></code>	Atributo <code>vertices</code> de Grafo
6	<code>ArrayList<Arista></code>	Atributo <code>aristas</code> de Grafo
7	Registrar Centro, Norte, Sur, Este y Oeste	<code>Main.cargarDatosIniciales</code>

#	Pauta	Dónde se cumple
8	Las 6 conexiones pedidas	<code>Main.cargarDatosIniciales</code>
9	Agregar vértices sin repetir nombres	<code>Grafo.agregarVertice</code> + menú opción 6
10	Agregar aristas entre vértices existentes	<code>Grafo.agregarArista</code> + menú opción 7
11	Buscar vértices por nombre	<code>Grafo.buscarVertice</code> + menú opción 2
12	Consultar y mostrar vecinos del vértice seleccionado	<code>obtenerVecinos, mostrarVecinos</code> + menú opción 3
13	Lista de adyacencia	<code>Grafo.mostrarListaAdyacencia</code> + menú opción 4
14	Recorridos for, for-each y forEach	Ver sección 6
15	Mensajes claros de error	agregarVertice, agregarArista, mostrarVecinos, buscar, menú
Reto	Distancia en km y mostrar rutas	<code>Arista.distanciaKm, Grafo.mostrarRutas</code>
Reto	Representación visual con Swing	<code>VentanaGrafo, PanelGrafo</code> + opción 8

Condición de entrega	Cómo se cumple
ArrayList como colección principal	Todos los datos del grafo están en dos ArrayList. No se usan mapas ni arreglos.
Conexiones como objetos, no texto	Cada conexión es un objeto Arista con referencias a dos objetos Vertice.
Encapsulamiento	Todos los atributos son private; los getters de Grafo devuelven copias; existeArista es privado; Vertice no tiene setter.
Separación de responsabilidades	modelo (datos), estructura (lógica), vista (dibujo), principal (menú).
Compila y ejecuta	Probado en NetBeans con JDK 21.

11. Preguntas frecuentes para la exposición

P: ¿Qué representa cada vértice?

R: Una zona de la ciudad donde la empresa de domicilios entrega pedidos. En el código es un objeto de la clase Vertice que guarda el nombre de la zona.

P: ¿Qué representa cada arista?

R: Una ruta directa entre dos zonas. Es un objeto Arista que guarda referencias a los dos objetos Vertice que une y la distancia en kilómetros.

P: ¿Cómo se almacenan en los ArrayList?

R: La clase Grafo tiene un ArrayList<Vertice> con las 5 zonas y un ArrayList<Arista> con las 6 rutas. Cada arista no copia los vértices: apunta a los mismos objetos que están en la lista de vértices.

P: ¿Por qué es un grafo no dirigido?

R: Porque las rutas se pueden recorrer en ambos sentidos. Por eso cada arista se guarda una sola vez y el método conecta() revisa los dos sentidos.

P: ¿Dónde está guardada la lista de adyacencia?

R: No está guardada: se calcula. Para cada vértice se recorren las aristas y se toma el otro extremo de las aristas que lo contienen (método obtenerVecinos).

P: ¿Cómo se evita un vértice repetido?

R: Antes de agregar, agregarVertice llama a buscarVertice. Si ya existe una zona con ese nombre (sin importar mayúsculas), muestra un error y no lo agrega.

P: ¿Cómo se evita una conexión repetida?

R: agregarArista llama a existeArista, que revisa con conecta() si alguna arista ya une esas dos zonas en cualquier sentido.

P: ¿Qué pasa si una zona no existe?

R: buscarVertice devuelve null y el método que la llamó muestra un mensaje de error claro, sin que el programa se detenga.

P: ¿Por qué los atributos son private?

R: Por encapsulamiento: los datos solo se modifican a través de métodos que validan. Por ejemplo, nadie puede agregar a la lista una arista con una zona inexistente.

P: ¿Por qué getVertices() devuelve una copia?

R: Para que quien la reciba no pueda modificar la lista real del grafo y saltarse las validaciones.

P: ¿Por qué se sobrescribió equals en Vertice?

R: Para que dos vértices con el mismo nombre se consideren iguales. Lo usan conecta, contiene, obtenerOtroExtremo e indexOf.

P: ¿Qué ventaja tendría una matriz de adyacencia? ¿Por qué no se usó?

R: La matriz permite saber en un paso si dos vértices están conectados, pero ocupa $n \times n$ posiciones y tiene tamaño fijo. La actividad pide explícitamente ArrayList, que crece al agregar zonas nuevas.

Anexo. Código completo de Grafo y PanelGrafo

Grafo.java

```
package com.grafos.estructura;

import com.grafos.modelo.Arista;
import com.grafos.modelo.Vertice;
import java.util.ArrayList;

/**
 * Grafo no dirigido que almacena sus vértices y aristas en ArrayList.
 */
public class Grafo {

    private ArrayList<Vertice> vertices;
    private ArrayList<Arista> aristas;

    public Grafo() {
        vertices = new ArrayList<>();
        aristas = new ArrayList<>();
    }

    /**
     * Agrega un vértice nuevo evitando nombres repetidos.
     */
    public boolean agregarVertice(String nombre) {
        if (nombre == null || nombre.trim().isEmpty()) {
            System.out.println("Error: el nombre del vértice no puede estar vacío.");
            return false;
        }
        if (buscarVertice(nombre) != null) {
            System.out.println("Error: el vértice '" + nombre + "' ya existe.");
            return false;
        }
        vertices.add(new Vertice(nombre.trim()));
        System.out.println("Vértice '" + nombre.trim() + "' agregado.");
        return true;
    }

    /**
     * Busca un vértice por su nombre (recorrido con for-each).
     * Devuelve null si no existe.
     */
    public Vertice buscarVertice(String nombre) {
        if (nombre == null) {
            return null;
        }
        for (Vertice v : vertices) {
            if (v.getNombre().equalsIgnoreCase(nombre.trim())) {
                return v;
            }
        }
        return null;
    }

    /**
     * Agrega una arista entre dos vértices existentes.
     */
    public boolean agregarArista(String nombreOrigen, String nombreDestino, double distanciaKm) {
```

```

Vertice origen = buscarVertice(nombreOrigen);
Vertice destino = buscarVertice(nombreDestino);

if (origen == null) {
    System.out.println("Error: no se puede conectar, el vértice '" + nombreOrigen + "' no
        existe.");
    return false;
}
if (destino == null) {
    System.out.println("Error: no se puede conectar, el vértice '" + nombreDestino + "'
        no existe.");
    return false;
}
if (origen.equals(destino)) {
    System.out.println("Error: no se puede conectar un vértice consigo mismo (" +
        nombreOrigen + ").");
    return false;
}
if (distanciaKm <= 0) {
    System.out.println("Error: la distancia debe ser mayor que 0 km.");
    return false;
}
if (existeArista(origen, destino)) {
    System.out.println("Error: la conexión " + origen + " - " + destino + " ya existe.");
    return false;
}
aristas.add(new Arista(origen, destino, distanciaKm));
System.out.println("Conexión " + origen + " - " + destino + " (" + distanciaKm + " km)
    agregada.");
return true;
}

/**
 * Verifica si ya existe una arista entre a y b (recorrido con for clásico).
 */
private boolean existeArista(Vertice a, Vertice b) {
    for (int i = 0; i < aristas.size(); i++) {
        if (aristas.get(i).conecta(a, b)) {
            return true;
        }
    }
    return false;
}

/**
 * Devuelve la lista de vecinos de un vértice.
 * Si el vértice no existe, devuelve una lista vacía.
 */
public ArrayList<Vertice> obtenerVecinos(String nombre) {
    ArrayList<Vertice> vecinos = new ArrayList<>();
    Vertice v = buscarVertice(nombre);
    if (v == null) {
        return vecinos;
    }
    for (Arista a : aristas) {
        if (a.contiene(v)) {
            vecinos.add(a.obtenerOtroExtremo(v));
        }
    }
    return vecinos;
}

/**
 * Muestra en consola los vecinos de un vértice.
 */

```

```

    public void mostrarVecinos(String nombre) {
        Vertice v = buscarVertice(nombre);
        if (v == null) {
            System.out.println("Error: el vértice '" + nombre + "' no existe.");
            return;
        }
        ArrayList<Vertice> vecinos = obtenerVecinos(nombre);
        if (vecinos.isEmpty()) {
            System.out.println(v + " no tiene vecinos.");
        } else {
            System.out.println("Vecinos de " + v + ": " + vecinos);
        }
    }

    /**
     * Muestra la lista de adyacencia del grafo.
     */
    public void mostrarListaAdyacencia() {
        System.out.println("\n===== LISTA DE ADYACENCIA =====");
        for (Vertice v : vertices) {
            ArrayList<Vertice> vecinos = obtenerVecinos(v.getNombre());
            StringBuilder linea = new StringBuilder(v.getNombre() + " -> ");
            for (int i = 0; i < vecinos.size(); i++) {
                linea.append(vecinos.get(i).getNombre());
                if (i < vecinos.size() - 1) {
                    linea.append(", ");
                }
            }
            System.out.println(linea);
        }
    }

    /**
     * Muestra todas las rutas con origen, destino y distancia (recorrido con forEach).
     */
    public void mostrarRutas() {
        System.out.println("\n===== RUTAS DISPONIBLES =====");
        if (aristas.isEmpty()) {
            System.out.println("No hay rutas registradas.");
            return;
        }
        aristas.forEach(a -> System.out.println(
            "Origen: " + a.getOrigen()
            + " | Destino: " + a.getDestino()
            + " | Distancia: " + a.getDistanciaKm() + " km"));
    }

    /**
     * Muestra todos los vértices registrados.
     */
    public void mostrarVertices() {
        System.out.println("\n===== ZONAS (VÉRTICES) =====");
        vertices.forEach(v -> System.out.println("- " + v));
    }

    // Se devuelven copias para proteger las listas internas (encapsulamiento).
    public ArrayList<Vertice> getVertices() {
        return new ArrayList<>(vertices);
    }

    public ArrayList<Arista> getAristas() {
        return new ArrayList<>(aristas);
    }
}

```

PanelGrafo.java

```
package com.grafos.vista;

import com.grafos.estructura.Grafo;
import com.grafos.modelo.Arista;
import com.grafos.modelo.Vertice;
import java.awt.BasicStroke;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import java.awt.FontMetrics;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Point;
import java.awt.RenderingHints;
import java.awt.geom.QuadCurve2D;
import java.util.ArrayList;
import javax.swing.JPanel;

/**
 * Panel que dibuja el grafo: cada vértice es un círculo y cada arista una línea.
 */
public class PanelGrafo extends JPanel {

    private static final int RADIO = 30;
    private static final int CURVATURA = 90;

    private Grafo grafo;
    // Posición de cada vértice en pantalla (misma posición que en getVertices()).
    private ArrayList<Point> posiciones;

    public PanelGrafo(Grafo grafo) {
        this.grafo = grafo;
        this.posiciones = new ArrayList<>();
        setPreferredSize(new Dimension(600, 500));
        setBackground(Color.WHITE);
    }

    /**
     * Calcula las posiciones: zonas conocidas en su punto cardinal,
     * cualquier otra zona se ubica en círculo alrededor del centro.
     */
    private void calcularPosiciones(ArrayList<Vertice> vertices) {
        posiciones.clear();
        int cx = getWidth() / 2;
        int cy = getHeight() / 2;
        int d = Math.min(getWidth(), getHeight()) / 2 - RADIO - 20;

        for (int i = 0; i < vertices.size(); i++) {
            String nombre = vertices.get(i).getNombre().toLowerCase();
            switch (nombre) {
                case "centro":
                    posiciones.add(new Point(cx, cy));
                    break;
                case "norte":
                    posiciones.add(new Point(cx, cy - d));
                    break;
            }
        }
    }
}
```

```

        case "sur":
            posiciones.add(new Point(cx, cy + d));
            break;
        case "este":
            posiciones.add(new Point(cx + d, cy));
            break;
        case "oeste":
            posiciones.add(new Point(cx - d, cy));
            break;
        default:
            double angulo = 2 * Math.PI * i / vertices.size() + Math.PI / 4;
            posiciones.add(new Point(
                cx + (int) (d * Math.cos(angulo)),
                cy + (int) (d * Math.sin(angulo))));
    }
}

/**
 * Indica si el punto medio de una arista cae sobre algún vértice.
 */
private boolean pasaPorOtroVertice(int x, int y) {
    for (Point p : posiciones) {
        if (p.distance(x, y) < RADIO) {
            return true;
        }
    }
    return false;
}

@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g;
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);

    ArrayList<Vertice> vertices = grafo.getVertices();
    calcularPosiciones(vertices);

    // 1. Dibujar aristas (líneas + distancia)
    g2.setStroke(new BasicStroke(2));
    g2.setFont(new Font("SansSerif", Font.PLAIN, 12));
    FontMetrics fmAristas = g2.getFontMetrics();
    for (Arista a : grafo.getAristas()) {
        Point p1 = posiciones.get(vertices.indexOf(a.getOrigen()));
        Point p2 = posiciones.get(vertices.indexOf(a.getDestino()));
        int mx = (p1.x + p2.x) / 2;
        int my = (p1.y + p2.y) / 2;
        String etiqueta = a.getDistanciaKm() + " km";
        int ex = mx + 5;

        g2.setColor(Color.GRAY);
        if (pasaPorOtroVertice(mx, my)) {
            // La línea recta taparía otro vértice: se dibuja como curva.
            double largo = Math.hypot(p2.x - p1.x, p2.y - p1.y);
            int cx = mx + (int) (-(p2.y - p1.y) / largo * 2 * CURVATURA);
            int cy = my + (int) ((p2.x - p1.x) / largo * 2 * CURVATURA);
            g2.draw(new QuadCurve2D.Double(p1.x, p1.y, cx, cy, p2.x, p2.y));
            // Punto más alto de la curva (donde va la etiqueta)
            mx = (mx + cx) / 2;
            my = (my + cy) / 2;
            // Etiqueta a la izquierda para no chocar con otras líneas
            ex = mx - fmAristas.stringWidth(etiqueta) - 8;
        } else {

```

```

        g2.drawLine(p1.x, p1.y, p2.x, p2.y);
    }

    g2.setColor(new Color(180, 40, 40));
    g2.drawString(etiqueta, ex, my - 5);
}

// 2. Dibujar vértices (círculos + nombre)
g2.setFont(new Font("SansSerif", Font.BOLD, 12));
FontMetrics fm = g2.getFontMetrics();
for (int i = 0; i < vertices.size(); i++) {
    Point p = posiciones.get(i);
    g2.setColor(new Color(30, 90, 160));
    g2.fillOval(p.x - RADIO, p.y - RADIO, RADIO * 2, RADIO * 2);
    g2.setColor(Color.WHITE);
    String nombre = vertices.get(i).getNombre();
    g2.drawString(nombre, p.x - fm.stringWidth(nombre) / 2, p.y + fm.getAscent() / 2 - 2);
}
}
}

```